

Auditoría Técnica del Pipeline GNN (Tab a Tab)

Documentación detallada, GraphSMOTE e ImGAGN

24 de febrero de 2026

Índice

1. Alcance y metodología	2
2. Mapa general del pipeline	3
3. Pipeline <i>tab a tab</i> (Streamlit Graph Builder)	3
3.1. Eventos	3
3.2. Feature Engineering	3
3.3. Feature Explorer	4
3.4. Feature Selection	4
3.5. Graph	4
3.6. Visual Graph	4
3.7. Network	4
3.8. Optuna	4
3.9. Balance	4
3.10. Training	5
3.11. Evaluación	5
3.12. History	5
3.13. Experiments	5
4. Modelo de datos del grafo	5
4.1. Aristas y atributos	5
4.2. Etiquetado temporal	5
5. Arquitectura del modelo GNN	6
5.1. Temporal head (opcional)	6
5.2. Registro de variantes GNN (encoder + temporal + aristas)	6
5.2.1. Idea general de comparabilidad	6
5.2.2. Variantes implementadas	7
5.2.3. Codificación de aristas (edge_attr)	7
5.2.4. Heads temporales y ensamblado de secuencias	7
5.2.5. Estrategias de cache temporal (punto crítico)	7
5.2.6. Matriz experimental recomendada	8
5.2.7. Métricas para eventos raros	8
5.2.8. Trazabilidad de variantes	8

6. Entrenamiento y optimización	8
6.1. Flujo del entrenamiento	8
6.2. Train-minibatch y regularización	9
7. GraphSMOTE: síntesis de nodos y aristas	9
7.1. Paso 1: embeddings	9
7.2. Paso 2: SMOTE en embeddings	9
7.3. Paso 3: decodificadores $z \rightarrow x$	9
7.4. Paso 4: inserción de nodos	10
7.5. Paso 5: generación de aristas sintéticas	10
7.6. Modos de operación	10
8. ImGAGN: generación adversarial	10
8.1. Paso 1: homogenización	10
8.2. Paso 2: definir cantidad de sintéticos	10
8.3. Paso 3: generador	11
8.4. Paso 4: discriminador	11
8.5. Paso 5: entrenamiento adversarial	11
8.6. Paso 6: grafo final	11
9. Optuna (búsqueda de hiperparámetros)	11
10. Evaluación, calibración y artefactos	12
11. Auditoría técnica y mejores prácticas	12
11.1. Fortalezas observadas	12
11.2. Riesgos o puntos críticos (actualizado)	12
11.3. Recomendaciones priorizadas (estado actual)	13
12. Anexo: referencias a funciones y archivos	13

1. Alcance y metodología

Este informe documenta y audita el pipeline GNN implementado en el repositorio, con foco en:

- El flujo *tab a tab* de la aplicación Streamlit (`src/graph_builder_app.py`).
- La construcción del grafo, el modelo GNN, el entrenamiento y la evaluación (`src/graph.py`, `src/gat_model.py`, `src/gnn_main.py`, `src/train_pretrain.py`).
- Los mecanismos de balanceo: GraphSMOTE e ImGAGN (`src/graphsmote.py`, `src/imgagn.py`).
- La búsqueda de hiperparámetros (Optuna) y el control de métricas.

La auditoría incluye verificación de mejores prácticas, riesgos de fuga de información, uso de memoria/cálculo, y consistencia con las opciones de configuración (`src/config.py`).

2. Mapa general del pipeline

Resumen conceptual:

1. **Datos base:** pórticos, flujos y accidentes (`src/utils.py`).
2. **Feature engineering:** snapshot/flow (`src/snapshot_pipeline.py`, `src/features.py`).
3. **Feature selection:** selección manual/guiada (UI) y persistencia.
4. **Grafo:** nodos `pm`, aristas `temporal/spatial/st_fwd` y atributos (`src/graph.py`).
5. **Entrenamiento:** HeteroGAT + TemporalAggregator (opcional) (`src/gat_model.py`, `src/temporal_head.py`).
6. **Balanceo:** GraphSMOTE (embedding-space + $z \rightarrow x$ + edge-gen) y/o ImGAGN (adversarial) (`src/graphsmote.py`, `src/imgagn.py`).
7. **Evaluación:** métricas (F1, AUC, AUPRC, MCC), umbrales y calibración (`src/gnn_main.py`).
8. **Artefactos:** modelos, hparams JSON, grafos aumentados, historial (`Resultados/`, `gnn_history.jsonl`).

3. Pipeline *tab a tab* (Streamlit Graph Builder)

El flujo UI se implementa en `src/graph_builder_app.py` y organiza el trabajo en tabs.

3.1. Eventos

Función UI: `_render_events_tab`.

Objetivo: cargar y validar accidentes, normalizar columnas, limpiar y preparar el dataset de eventos.

- Carga de accidentes desde CSV/archivos seleccionados, persistencia en `st.session_state`.
- Uso de utilidades en `src/utils.py` para normalizar columnas y fechas.
- Soporte para generar reportes de *Puntos Negros* (posteriormente usados en *Graph*).

Implicación: este tab define el universo de accidentes, que afectará etiquetas futuras y cortes temporales.

3.2. Feature Engineering

Función UI: `_render_feature_engineering`.

Objetivo: generar o cargar features por nodo `pm`.

- Modos principales: *Snapshot Features* y *Flow 5 min* (ver `src/snapshot_pipeline.py`).
- Generación de features con `compute_pm_features` (`src/features.py`) y/o `SnapshotFeatureBuilder`.
- Salida persistida en DuckDB o CSV; sincronización de rango temporal y tramo.

Implicación: afecta dimensionalidad de nodos y distribución de atributos. La resolución temporal se gobierna con `DT_MINUTES` y parámetros en `src/config.py`.

3.3. Feature Explorer

Función UI: `render_feature_explorer`.

Objetivo: inspección visual, estadísticas y outliers de variables de entrada. **Implicación:** diagnóstico de calidad de features antes de construir el grafo.

3.4. Feature Selection

Función UI: `_render_feature_selection_tab`.

Objetivo: seleccionar subconjuntos de variables para entrenamiento, guardar listas y metadatos.

Implicación: cambia el espacio de entrada del GNN y la dimensionalidad de `pm.x`.

3.5. Graph

Funciones UI: `_render_create_graph`, `_render_load_graph`, `_render_in_memory_graph`.

Objetivo: construir o cargar el grafo heterogéneo de pórticos.

- Selección de tramo (manual o por puntos negros).
- Construcción de nodos `pm` (pórtico-minuto) con features normalizados.
- Generación de aristas temporales/espaciales y atributos de arista (ver Sección 4).
- Creación de `train/val/test_mask` con corte temporal (`_compute_time_cutoffs`).
- Guardado en `Resultados/highway_graph_*.pt` y en memoria (`st.session_state`).

3.6. Visual Graph

Función UI: `render_visual_graph_tab`.

Objetivo: visualizar topología, grados, y subgrafos para QA.

3.7. Network

Función UI: `_render_network_tab`.

Objetivo: configurar arquitectura y parámetros del modelo antes de entrenar. **Implicación:** fija opciones como `hidden_channels`, `num_heads`, `dropout`, `num_layers`, agregaciones `aggr1/aggr2` y checkpointing.

3.8. Optuna

Función UI: `_render_optuna_tab`.

Objetivo: búsqueda de hiperparámetros con Optuna (ver Sección 9). **Implicación:** almacena `optuna_hyperparams_*.csv` y gobierna la selección automática en entrenamiento.

3.9. Balance

Función UI: `_render_balance_tab`.

Objetivo: generar grafos balanceados con GraphSMOTE o ImGAGN.

- GraphSMOTE: requiere modelo entrenado para embeddings y decodificadores $\mathbf{z} \rightarrow \mathbf{x}$.
- ImGAGN: entrena generador y discriminador adversarial y crea nodos sintéticos.

Implicación: genera grafos con `is_synthetic`, actualiza `train_mask`, y puede cambiar el balance real.

3.10. Training

Función UI: `_render_training_tab`.

Objetivo: entrenar el GNN con opción de grafo balanceado, early stopping y evaluación automática.

Backend: `src/gnn_main.py::run_gat_training` y `src/train_pretrain.py::train_minibatch`.

3.11. Evaluación

Función UI: `_render_evaluation_tab`.

Objetivo: cargar modelos, evaluar métricas, aplicar umbral y calibración opcional. **Backend:**

`src/gnn_main.py::test` y utilidades de exportación.

3.12. History

Función UI: `_render_history_tab`.

Objetivo: registro de experimentos y trazabilidad (`Resultados/gnn_history.jsonl`).

3.13. Experiments

Función UI: `_render_gnn_experiments_tab`.

Objetivo: ejecutar lotes experimentales, exportar resultados, e importar ZIPs.

4. Modelo de datos del grafo

Nodos: `pm` (pórtico-minuto).

Targets: `pm.y` (etiqueta binaria), `pm.is_accident_pm` (accidente directo).

Máscaras: `train_mask/val_mask/test_mask/sequence_mask`.

4.1. Aristas y atributos

Aristas principales (`src/graph.py`):

- **temporal:** $P_{i,t} \rightarrow P_{i,t+1}$ (misma ubicación, siguiente tiempo).
- **spatial:** $P_{i,t} \rightarrow P_{i+1,t}$ (siguiente pórtico, mismo tiempo).
- **st_fwd** (opcional): $P_{i,t} \rightarrow P_{i+1,t+1}$.
- **spatial_back** (opcional): inversa de **spatial**.

Atributos de arista: diferencia de features entre destino y origen (Δx) y extras para **spatial** (gradientes, shock, distancia). En **temporal** y **st_fwd** se usan *extras* en cero para alinear dimensiones.

4.2. Etiquetado temporal

Cortes temporales: definidos por accidentes ordenados (`_compute_time_cutoffs`).

Máscaras: aplicadas por timestamps en minutos.

SequenceIndex: secuencias temporales con guard band y horizonte futuro (`src/snapshot_sequences.py`).

5. Arquitectura del modelo GNN

Modelo base: `src/gat_model.py::HeteroGAT`.

- **Capas:** `num_layers` bloques `HeteroConv` con `GATConvSaveAlpha`.
- **Agregación:** `aggr1` en la primera capa, `aggr2` en capas siguientes.
- **Atenciones:** capturadas cuando `XAI=1`, usadas para regularización $L2$.
- **Edge attributes:** se validan por batch; si faltan o desalinean, se rellenan con ceros.
- **Checkpointing:** `use_checkpointing` reduce memoria con recomputación.

5.1. Temporal head (opcional)

Clase: `src/temporal_head.py::TemporalAggregator`.

Idea: GRU sobre secuencias de embeddings recientes; mantiene cache de embeddings para nodos no presentes en el batch, evitando fuga de gradientes.

5.2. Registro de variantes GNN (encoder + temporal + aristas)

El pipeline ahora incorpora un **registro de variantes** controlado por `src/config.py` (`GNN_VARIANT`, `GNN_TEMPORAL_CACHE_STRATEGY`) y materializado en `src/gnn_main.py` mediante las funciones `_build_gnn_model`, `_build_temporal_head_for_variant` y `_prime_temporal_cache_if_needed`. Además, la UI en `src/graph_builder_app.py` expone una guía resumida en los tabs *Network*, *Training*, *Optuna* y *Evaluación*. La idea de diseño es separar:

- **Encoder espacial:** GAT heterogéneo base, GAT con codificador de aristas, o variante edge-aware.
- **Head temporal:** snapshot (sin secuencia), GRU, Transformer temporal o attention pooling.
- **Protocolo de cache:** incremental por batch o `global_epoch` (forward global por época para secuencias consistentes).

5.2.1. Idea general de comparabilidad

Todas las variantes comparten el mismo `SequenceIndex` (construido en `src/snapshot_sequences.py` a partir de `sequence_rows/target_rows`), el mismo grafo y los mismos splits temporales. Esto permite comparar arquitecturas sin cambiar la definición de etiqueta ni el horizonte de predicción.

Etiquetado “5 minutos antes” La lógica de etiquetado usa:

$$\text{intervalo positivo} = (t + \gamma, t + \gamma + H]$$

donde γ es `GUARD_BAND_MINUTES` y H es `HORIZON_MINUTES`. Para una configuración estricta de “5 minutos antes”, se recomienda:

- `GUARD_BAND_MINUTES` = 5
- `HORIZON_MINUTES` = 5 (o 10–20 para riesgo en ventana)

en `src/config.py`.

5.2.2. Variantes implementadas

Implementación: `src/gat_model.py`, `src/temporal_head.py`, `src/temporal_heads.py`, `src/gnn_main.py`.

Selección: mediante el string `GNN_VARIANT` (normalizado en `src/gnn_main.py::_normalize_gnn_variant`).

1. `gat_snapshot` (baseline): HeteroGAT sin head temporal. Clasifica con el embedding del snapshot actual.
2. `gat_gru`: HeteroGAT + `TemporalGRUHead` (`src/temporal_heads.py`).
3. `gat_transformer`: HeteroGAT + `TemporalTransformerHead` (self-attention con positional embedding aprendible).
4. `gat_attnpool`: HeteroGAT + `TemporalAttnPoolHead` (ablation ligera y barata computacionalmente).
5. `gat_edge_mlp`: `HeteroGATWithEdgeEncoder` (MLP por tipo de arista sobre `edge_attr`) sin head temporal.
6. `gat_edge_mlp_gru` y `gat_edge_mlp_transformer`: combinan codificador de aristas + head temporal.
7. `gnn_edge_aware`: `HeteroEdgeAware` (usa `TransformerConv`) sin head temporal.
8. `gnn_edge_aware_gru` y `gnn_edge_aware_transformer`: variante edge-aware con modelado temporal.

5.2.3. Codificación de aristas (`edge_attr`)

La variante `HeteroGATWithEdgeEncoder` agrega un MLP de aristas (`EdgeAttrEncoder`) antes de la atención. Motivación:

- los deltas Δx pueden tener escalas heterogéneas;
- un mapeo aprendido puede inducir una geometría más estable para la atención;
- permite estudiar de forma separada el efecto de “mejor `edge_attr`” vs. “mejor head temporal”.

5.2.4. Heads temporales y ensamblado de secuencias

Los heads temporales reutilizan la infraestructura de `TemporalAggregator`:

- `update_cache(...)` para cache global o parcial de embeddings.
- `_build_sequence_batch(...)` para reconstruir tensores $[B, L, D]$ usando `sequence_rows`.

Esto permite cambiar el operador temporal (GRU/Transformer/Attention Pooling) sin cambiar el contrato del entrenamiento en `src/train_pretrain.py` ni la firma de `forward`.

5.2.5. Estrategias de cache temporal (punto crítico)

- `incremental`: cache por batch (histórico, menor costo, más dependiente del sampler).
- `global_epoch`: priming por época con forward global (`compute_epoch_embeddings`), luego training/eval usan cache consistente.

Recomendación para artículo/ciencia: usar `global_epoch` para que las secuencias temporales tengan la misma referencia de embeddings dentro de una época.

5.2.6. Matriz experimental recomendada

1. `gat_snapshot` (baseline espacial)
2. `gat_gru`
3. `gat_transformer`
4. `gat_edge_mlp_gru`
5. `gnn_edge_aware_gru` (o `gnn_edge_aware_transformer`)

Mantener semilla, split temporal, `SequenceIndex` y protocolo de evaluación constantes.

5.2.7. Métricas para eventos raros

Para comparación científica y operativa, priorizar:

- **PR-AUC / AUPRC**: métrica principal en desbalance severo.
- **ROC-AUC**: referencia secundaria.
- **Recall@FAR/FPR objetivo**: útil para restricciones operacionales.
- **MCC**: robusta cuando hay fuerte asimetría de clases.

5.2.8. Trazabilidad de variantes

El entrenamiento y HPO guardan `gnn_variant` y `variant_tag` en sidecars de hiperparámetros, checkpoints y logs estructurados (`Resultados/gat_model_BEST*.pt`, `_hparams.json`, Optuna CSVs). Esto evita mezclar checkpoints de arquitecturas incompatibles.

6. Entrenamiento y optimización

Punto de entrada: `src/gnn_main.py::run_gat_training`.

6.1. Flujo del entrenamiento

1. **Dispositivo**: `get_auto_device()` (MPS > CUDA > CPU).
2. **Carga de datos**: `loaded_obj['data']`.
3. **ImGAGN automático** (si `AUTO_IMGAGN_PRETRAIN=1`).
4. **GraphSMOTE**: elección por usuario/flag y detección de sintéticos existentes.
5. **HPO**: selección de `optuna_hyperparams_*.csv` o búsqueda nueva.
6. **Modelo**: fábrica `_build_gnn_model` selecciona encoder/temporal head según `gnn_variant` y adjunta el head temporal cuando aplica.
7. **Decodificadores $z \rightarrow x$** : entrenados si GraphSMOTE activo (`train_z2x_decoders`).
8. **Edge generator**: `RelEdgeGen` para reconstrucción y aristas sintéticas.

9. **Pérdida:** CrossEntropy o FocalLoss (con/ sin pesos).
10. **Scheduler:** OneCycleLR.
11. **Loop:** entrenamiento por mini-batches, validación, early stopping, guardado de mejor modelo.

6.2. Train-minibatch y regularización

Función: `src/train_pretrain.py::train_minibatch`.

- Clasificación sobre pm (primeros `batch_size` del `NeighborLoader`).
- Pérdida total:

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_{edge} \mathcal{L}_{edge} + \lambda_{att} \mathcal{L}_{att}.$$

- $\mathcal{L}_{edge}$: reconstrucción de aristas con muestreo negativo (si `edge_gen`).
- $\mathcal{L}_{att}$: L2 sobre atenciones cuando `XAI=1`.
- Gradiente acumulado (`accumulation_steps=2`) y `grad_clip`.
- Soporte AMP (solo CUDA).

7. GraphSMOTE: síntesis de nodos y aristas

Módulo: `src/graphsmote.py`.

Principio: generar nodos sintéticos en el espacio embedding y decodificarlos a features reales.

7.1. Paso 1: embeddings

- `get_embeddings_minibatch` (para balance offline en UI) o `compute_train_embeddings` (en entrenamiento, sin fuga).
- Normalización ℓ_2 por nodo.

7.2. Paso 2: SMOTE en embeddings

Se aplica SMOTE en el conjunto minoritario:

$$\mathbf{z}_{syn} = (1 - \alpha) \mathbf{z}_i + \alpha \mathbf{z}_j, \quad \alpha \sim U(0, 1).$$

- k-NN sobre embeddings minoritarios (CPU cache o GPU si cabe).
- Parámetros clave: `GRAPHSMOTE_K`, `smote_k`, `random_state`.

7.3. Paso 3: decodificadores $\mathbf{z} \rightarrow \mathbf{x}$

Clase: `Z2XDecoders`.

Entrenamiento: `train_z2x_decoders` con SmoothL1/MSE, early stopping y validación.

- Un MLP por tipo de nodo: $\mathbf{x}_{syn} = f_{ntype}(\mathbf{z}_{syn})$.
- Se respetan `train/val_mask` y se evita usar nodos fuera del split cuando está disponible.

7.4. Paso 4: inserción de nodos

- Concatenar `x` y `y` sintéticos a `pm.x` y `pm.y`.
- Actualizar `train/val/test_mask` (por defecto nuevos nodos van a `train`).
- Crear/actualizar `is_synthetic` y `is_accident_pm`.

7.5. Paso 5: generación de aristas sintéticas

Generador: `RelEdgeGen` (parámetro por relación).

Candidatos reales:

- `GRAPHSMOTE_CONNECT=1`: top-K en `train_mask`.
- `GRAPHSMOTE_CONNECT=0`: vecinos a 1 salto de accidentes reales.

Dirección:

- `BIDIRECTION=1`: real $\leftrightarrow$ sintético.
- `BIDIRECTION=0`: solo real $\rightarrow$ sintético.

Atributos de arista: se computan como Δx entre nodos destino y origen (con `delta_feature_idx` si existe) y se alinean a la dimensionalidad existente.

7.6. Modos de operación

- **Offline:** `augment_graph_offline_once` aumenta una vez y usa el grafo para entrenar.
- **Online:** `refresh_synthetics_online` regenera nodos cada `SMOTE_EVERY_N_EPOCHS`.
- **None:** sin aumentación.

8. ImGAGN: generación adversarial

Módulo: `src/imgagn.py`.

Idea central: generar nodos sintéticos como combinaciones convexas de nodos minoritarios *de entrenamiento* y entrenar un discriminador adversarial.

8.1. Paso 1: homogenización

`to_homogeneous_if_needed` transforma `HeteroData` a grafo homogéneo (target `pm`), preservando `edge_attr` y `delta_feature_idx` si existen.

8.2. Paso 2: definir cantidad de sintéticos

$$\lambda_1 = \frac{n_{min} + n_g}{n_{maj}}$$
$$\Rightarrow n_g = \text{máx}(0, \lambda_1 n_{maj} - n_{min})$$

Se limita con `max_new_nodes` para evitar OOM.

8.3. Paso 3: generador

Clase: `ImGAGNGenerator`.

- Entrada: ruido $\mathbf{z} \sim \mathcal{N}(0, I)$.
- Salida: logits por nodo minoritario de entrenamiento.
- Pesos $T = \text{softmax}(G(z))$ y features sintéticos $X_g = TX_{min}$.
- Aristas: top-K conexiones a nodos minoritarios reales según T .

8.4. Paso 4: discriminador

Clase: `ImGAGNDiscriminator` (GCN + 2 cabezas).

- **Head real/fake** (BCE).
- **Head minority/majority** (BCE).
- Término de separación `_mm_separation` para alejar medias minoritaria y mayoritaria.

8.5. Paso 5: entrenamiento adversarial

Por cada `epoch`:

1. Muestrear ruido y construir sintéticos (sin gradientes).
2. Construir grafo aumentado y entrenar D por `d_steps` minibatches.
3. Recalcular X_g con gradientes y optimizar G para:

$$\mathcal{L}_G = \mathcal{L}_{rf} + \mathcal{L}_{mi} + \mathcal{L}_{di},$$

donde $\mathcal{L}_{rf}$ empuja a *real*, $\mathcal{L}_{mi}$ empuja a *minority*, y $\mathcal{L}_{di}$ acerca embeddings a centroides minoritarios.

Escalabilidad: usa `NeighborLoader` si está disponible; si no, full-batch.

8.6. Paso 6: grafo final

Se reconstruye el grafo aumentado con el generador final y se devuelven: `x_aug`, `edge_index_aug`, `edge_attr_aug`, y la porción de nodos generados.

9. Optuna (búsqueda de hiperparámetros)

Función: `src/gnn_main.py::objective`.

- Objetivo: maximizar F0.5 en validación (prioriza precisión).
- Espacio: `hidden_channels`, `num_heads`, `dropout`, `num_layers`, `aggr1/aggr2`.
- Optimizadores: Adam/AdamW/RAdam (configurable).

- Con GraphSMOTE: busca `smote_k`, `target_pos_ratio`, `smote_every_n_epochs`, `lambda_edge`.
- Sin GraphSMOTE: FocalLoss con `focal_alpha`, `focal_gamma`.

Muestreo de seeds (train/val):

- *Resamplear seeds por epoch*: cambia el subset en cada epoch (mide robustez).
- *Fijar subset por trial*: mantiene el mismo subset durante el trial (reduce varianza).
- *Cargar todo el grafo en memoria (sin muestreo)*: usa todos los nodos `train/val` (equivale a `sample_train_nodes=0` y `sample_val_nodes=0`); mayor uso de memoria.

10. Evaluación, calibración y artefactos

- **Evaluación**: `test` calcula F1, AUC, AUPRC, MCC, matriz de confusión.
- **Umbral**: `pick_tau_fbeta` busca umbral τ en validación.
- **Calibración**: Platt scaling opcional (`AUTOCALIBRATE_PROBS`).
- **Guardado**: modelos `gat_model_BEST*.pt`, `GraphSMOTE_embeddings_model*.pt`, `sidecar_hparams.json`.
- **Hash del grafo**: `_calculate_graph_hash` para trazabilidad.

11. Auditoría técnica y mejores prácticas

11.1. Fortalezas observadas

- Separación temporal para `train/val/test` y uso de `sequence_mask` reduce fuga temporal.
- GraphSMOTE en entrenamiento usa embeddings de `train` (`compute_train_embeddings`).
- `NeighborLoader` limita memoria y permite entrenamiento en grafos grandes.
- Mecanismos de limpieza/*coalesce* para evitar inconsistencias en `edge_attr`.
- Guardado de `hparams` y `run_id` para reproducibilidad.

11.2. Riesgos o puntos críticos (actualizado)

- **Fuga en balanceo manual (mitigado)**: `run_graphsmote` ahora restringe embeddings/pool minoritario a `train_mask`. Mantener verificación en auditorías.
- **Dependencia de BIDIRECTION**: dirección de aristas sintéticas afecta mensajes y puede sesgar la difusión de señales. Se mantiene sin cambios por ahora.
- **Acumulación de gradientes (ajustable)**: `accumulation_steps` ahora es configurable (config/UI). Aún requiere tuning según memoria/hardware.
- **Balanceo + pesos de clase**: se deshabilita pesos con GraphSMOTE/ImGAGN (correcto), pero requiere monitoreo de métricas en validación.

11.3. Recomendaciones priorizadas (estado actual)

1. **Implementado:** en `run_graphsmote` (balance manual) se limita a `train_mask` para evitar fuga potencial.
2. **Implementado:** exponer `accumulation_steps` en `config/UI`; ajustar según memoria disponible.
3. **Pendiente/decisión:** no modificar `BIDIRECTION` por ahora; documentar el racional y evaluar impacto en AUPRC/F1 si se cambia en el futuro.
4. **Implementado:** en `ImGAGN` registrar `best_train_recall` junto a `seed` y `config` para trazabilidad.

12. Anexo: referencias a funciones y archivos

- UI principal: `src/graph_builder_app.py`.
- Construcción del grafo: `src/graph.py`.
- Modelo GNN: `src/gat_model.py`.
- Temporal head: `src/temporal_head.py`.
- Entrenamiento/evaluación: `src/gnn_main.py`, `src/train_pretrain.py`.
- GraphSMOTE: `src/graphsmote.py`.
- ImGAGN: `src/imgagn.py`.
- Configuración: `src/config.py`.